

PLAIN-ENGLISH GUIDE

How I Built This Project

Every single step explained in plain English - what I did, why I did it, and what else I could have done instead

Author	bann
Document	Beginner's guide booklet
Companion to	Equity Research Report and Project Notes
Written for	Someone who has never built a financial model, written a line of SQL, or opened Power BI
Promise	No step is skipped and no jargon is used without being explained the first time it appears



How to use this booklet

This is the instruction manual for the whole project. If you wanted to rebuild everything from scratch, you could do it by following this document from top to bottom. Nothing is assumed. Every technical word gets explained the first time it turns up, in a shaded box like this one.

Jargon box: what is a 'sovereign wealth fund'? A country's savings account, run like an investment firm. Instead of one family investing its money, it is an entire government. Mubadala is Abu Dhabi's. It takes money the country earned (historically from oil) and buys businesses, buildings and shares all over the world, so that future generations still have income when the oil runs out. That single sentence explains almost everything unusual about how it invests.

Each of the nine steps below follows the same shape:

- **What I did** - the actual actions, in order.
- **Why I did it that way** - the reasoning.
- **What else I could have done** - a table of the realistic alternatives, with honest pros and cons.
- **What went wrong** - because something did, in almost every step, and the fixes are the part worth reading.
- **What to say in an interview** - the one or two sentences that turn the work into something a recruiter can grade.

The most important idea in this whole booklet. A finance project is not impressive because the numbers are big. It is impressive because a stranger can check every number you wrote down. Almost every choice in this project was made in favour of being checkable rather than being impressive. Recruiters can tell the difference immediately, and it is the difference between a project that gets you an interview and one that gets you caught out in it.

0

Deciding what the project actually is

Before touching data, decide what question you are answering

What I did

The project brief was three lines long: research Mubadala's portfolio strategy and sector allocation, build valuation case studies in four sectors, and build dashboards and datasets. That is a topic, not a question. So the first thing I did was write down five questions the finished project would answer:

1. Is Mubadala's portfolio concentrated or spread out, and does it matter?
2. Is the fund growing because it is being given money, or because it is making money?
3. Is the mix of assets it holds actually a sensible mix?
4. How important is technology to the overall strategy, really?
5. Are the individual companies it owns cheap, expensive, or about right?

Why I did it that way

A project organised around questions has a natural ending: you stop when the questions are answered. A project organised around a topic never ends, because there is always one more chart you could make. It also changes what you build. If question three is 'is the mix sensible', you now *need* a portfolio optimiser, and you know exactly what it is for. Without the question, the optimiser is just a thing you added because it looked clever - and an interviewer will find that out in about thirty seconds by asking 'why did you do that?'

Option	Good things about it	Bad things about it
Start with a topic	Easy to begin. You can start reading immediately without deciding anything.	No natural end. Easy to produce a lot of material that proves nothing. Very hard to write a conclusion.
Start with questions	Every piece of work has an obvious purpose. The project has a clear end point. Findings write themselves.	Slower to start. You may have to change the questions once you see what data actually exists.
Start with the tools	You get to practise the software straight away, which feels productive.	You end up with a dashboard that shows things nobody asked about. Classic beginner trap - lots of charts, no argument.

What I picked and why. Questions. It cost me about an hour up front and saved days later, because every time I was unsure whether to build something, I asked 'which of my five questions does this answer?' If the answer was 'none', I did not build it.

Interview line. "I framed it as five questions rather than a topic, so every component in the repository exists to answer one of them. That is also why there is no chart in the project that I can't explain the purpose of."

1

Finding real information

Going to the source instead of trusting a summary

What I did

I went to Mubadala's own website and read the primary documents rather than news articles about them:

- The **2025 annual results press release**, published 9 April 2026 - this is where the headline numbers live.
- The **2025 Annual Review** microsite, especially the Performance Overview page, which has the asset-class breakdown.
- The **Our Structure** page, which describes the four investment platforms and names the people who run them.
- The **newsroom**, which lists every deal announcement - this became the transaction dataset.
- For the listed companies, **StockAnalysis.com**, which republishes S&P Global Market Intelligence data.

Here are the numbers that came out of that, and they are the spine of everything else:

Number	Value	Where it came from
Assets under management	AED1.414 trillion (US\$385bn)	Mubadala press release, 9 Apr 2026
Growth in the year	+17%	Same
5-year annualised return	10.7%	Same
10-year annualised return	10.3%	Same
Money invested in 2025	US\$39bn (AED143bn)	Same
Money received back in 2025	US\$38bn (AED138bn)	Same
Asset mix	Private 42%, Public 20%, Real estate & infra 17%, Alternatives 16%, Credit 5%	2025 Annual Review

Jargon box: AED and the peg. AED is the UAE dirham. It is *pegged* to the US dollar at a fixed rate of 3.6725 dirhams per dollar, and has been since 1997. A peg means the exchange rate does not float around - the central bank commits to holding it. That is why you can convert every dirham figure in this project to dollars with a single constant and not worry about which day you did it.

Why I did it that way

Two reasons. First, accuracy: every time a number passes through a journalist it can be rounded, mistranslated or given without context. Second, and more useful for you: **a recruiter can click your source**. If your footnote says 'Mubadala press release, 9 April 2026' they can verify it in ten seconds. If it says 'various media reports' they cannot, and they will assume the worst.

Option	Good things about it	Bad things about it
Company primary sources	Free. Authoritative. Precisely dated. The company is legally careful about what it publishes.	Selective - a company publishes what flatters it. No position-level portfolio is ever disclosed.
News articles	Fast. Often adds useful context and quotes that the company itself would not give.	Errors propagate. Numbers get rounded and lose their as-of date. Often no link back to the original.
A paid database (Bloomberg, PitchBook)	Complete, structured, includes private-company estimates. What the industry actually uses.	Extremely expensive. Licences forbid republishing, so you often cannot show your workings anyway.

What I picked and why. Primary sources for anything about Mubadala, and StockAnalysis (which republishes S&P Global data) for share prices and multiples. This combination is free, citable and reproducible, which matters more for a portfolio project than completeness does.

What went wrong

The Annual Review's geography breakdown is drawn by JavaScript in the browser. When I fetched the page I got the labels - North America, Europe, UAE, Asia-Pacific - but every percentage came back as '0%'. Rather than guess the numbers or quietly drop the analysis, I built my own geographic picture from the twenty named holdings and **labelled it as my dataset, not Mubadala's disclosure**. The report says so explicitly.

This is the single most transferable lesson in the booklet. When you cannot get a number, you have three options: guess it, hide the gap, or say so and work around it. The first two are how people get found out. The third is how you build a reputation. Every analyst hits missing data constantly - the job is not to have perfect data, it is to be honest about which data you have.

2

Organising the information into a database

Turning scattered facts into something a computer can slice

What I did

I built sixteen tables and loaded them into a SQLite database. The tables come in two types, and this split is called a **star schema**.

Jargon box: star schema, dimension, fact. Imagine a school. A **dimension** table is a list of things that barely change: the pupils, the subjects, the teachers. A **fact** table is a list of things that happen: 'pupil 14 scored 78% in subject 3 on 12 March'. The fact table only stores ID numbers, and you look up the names in the dimension tables. Drawn out, the facts sit in the middle and the dimensions radiate outward like the points of a star - hence the name. It is the standard way to build data for reporting tools.

Dimension tables (the 'nouns')	Fact tables (the 'events and measurements')
dim_sector - the 12 sectors dim_geography - 12 countries and their regions dim_platform - Mubadala's 4 investment platforms dim_asset_class - the 5 asset classes dim_source - every source I used, with its URL and date	fact_performance - AUM, returns, deployments fact_capital_flow - money in and out by year fact_holding - 20 named holdings fact_transaction - 19 announced deals fact_market_data - 20 listed companies' prices and multiples fact_gfs_financials / fact_omv_financials - company accounts fact_whoop_metrics - WHOOP's disclosed numbers

Why I did it that way

The obvious alternative is one big spreadsheet with every column in it. That works until you want to ask a question. Suppose you type 'United States' in one row and 'USA' in another - now a filter on country misses half your data, silently. In a star schema the country lives in exactly one place and every other table points at it by ID, so that mistake becomes impossible.

There is also a practical reason specific to this project: Power BI, Tableau and every other reporting tool are *built* for star schemas. One sector filter can control the holdings chart, the deals chart and the market-data chart at the same time, because all three point at the same dimension table. With one flat table you would need three separate filters and they would drift out of step.

Option	Good things about it	Bad things about it
One big spreadsheet	Anyone can open it. Zero learning curve. Fine for under a few hundred rows.	Typos create invisible errors. Text repeats on every row. Slicers become ambiguous. Does not scale.
Star schema	Filters behave predictably. Reporting tools are designed for it. No duplicated text.	You have to learn about joins and keys. Slightly more set-up work at the start.
Fully normalised (3NF)	Mathematically the tidiest. Zero redundancy anywhere. What transactional databases use.	Queries need many joins and become hard to read. Reporting tools run slower on it. Overkill here.

What I picked and why. Star schema. It is the industry standard for analytics precisely because it sits between the two extremes: tidy enough to prevent errors, simple enough that a query stays readable.

The one extra column that makes this project different

Every fact table has a **source_id** column and most have an **evidence_grade**. Grades are: *Official* (Mubadala said it), *Market data* (an exchange or data vendor said it), *Press reporting* (widely reported but not confirmed), and *Analyst estimate* (I made it up, with stated reasoning). Sixteen of the twenty holdings are Official and four are Press reporting, and the dashboard shows the grade next to every row.

Why this is worth doing even though nobody asked for it. It means that when someone challenges a number, you do not have to remember where it came from - you can run one query and show them. It also forces honesty on yourself while you work, because writing 'Analyst estimate' next to a figure makes it much harder to quietly treat a guess as a fact later on.

What went wrong

SQLite refused to create the database file, returning 'disk I/O error'. The cause was that the folder I was working in is a network-style mount that does not support the file locking SQLite needs. The fix was three lines: build the database on local disk, then copy the finished file into the project folder. The code still carries a comment explaining why, because otherwise the next person to read it would 'fix' it back.

3

Asking the database questions with SQL

Twelve queries that turn stored data into findings

Jargon box: SQL. SQL (say it 'sequel' or 'ess-cue-ell', both are accepted) stands for Structured Query Language. It is how you ask a database for things. It reads almost like English: SELECT the columns you want, FROM the table, WHERE some condition is true. That is 80% of it.

What I did

I wrote twelve queries, each deliberately using a different technique so the file demonstrates range as well as answering the questions.

Query	What it answers	Technique it shows off
Q2	How concentrated is the asset mix?	Window function with a running total
Q3	Is growth self-funded?	LAG() to compare a row to the previous year
Q4	Which sectors dominate?	GROUP BY with a percent-of-total subquery
Q6	Which platform invests in what?	Conditional aggregation (a pivot without PIVOT)
Q7	How has deal flow moved?	CTE plus date extraction and honest NULL handling
Q8	Herfindahl concentration index	Mathematical aggregate with CASE interpretation
Q9	How do peers compare on multiples?	PARTITION BY and NTILE for quartiles
Q10	How is GlobalFoundries' mix shifting?	Self-join of a CTE to compare two years
Q11	Is OMV's dividend safe?	Derived ratio with a CASE-based verdict
Q12	How much of my own data is officially sourced?	UNION ALL audit across tables

Two findings that came straight out of these queries

Q8 - the concentration index. The Herfindahl-Hirschman index adds up the squares of the percentage weights. Five equal buckets of 20% would give $5 \times 400 = 2,000$. Mubadala's actual mix gives 2,734. Anything above 2,500 is officially 'concentrated'. Flip the number over and you get an 'effective number of buckets' of 3.7 - so despite reporting five asset classes, the portfolio behaves like about three and a half.

Q11 - the dividend cover. Dividing OMV's free cash flow by the cost of its dividend gives 6.0x in 2021, then 5.3x, 2.3x, 2.0x and 1.3x in 2025. That is a company raising its dividend every year while the cash behind it shrinks. This was the single most useful finding in the whole project and it came from four lines of arithmetic in a SQL query.

Interview line. "The most valuable thing in my SQL file is four lines long. I divided OMV's free cash flow by its dividend cost across five years and the cover ratio had fallen from six times to one point three, while the dividend per share went up every single year. Nothing clever - just asking the obvious question of the data."

Option	Good things about it	Bad things about it
Do it all in Excel	Visual, instant feedback, everyone can open the file.	Formulas hide in cells. Impossible to review. One dragged formula corrupts everything silently.
Do it in SQL	The logic is written down and re-runnable. Handles joins naturally. Universally expected in finance data roles.	You must learn it, and debugging a long query is genuinely annoying at first.
Do it in Python pandas	Most flexible. Same language as the models and the machine learning.	Reviewers cannot check it without running it. Fewer finance interviewers read Python than read SQL.

What I picked and why. SQL for anything that is a question about the dataset, Python for anything that is a calculation. That split also happens to be how most real analytics teams work.

4

Working out what a company is worth

The four valuation case studies, explained from zero

The idea behind all valuation, in one paragraph

A business is worth the cash it will hand you in the future, adjusted for the fact that cash later is worth less than cash now. That is it. Everything else - every model, every multiple, every spreadsheet - is a different way of estimating those two things. If you understand that sentence you understand valuation; the rest is bookkeeping.

Jargon box: discounting, and why later money is worth less. If someone offers you 100 pounds today or 100 pounds in a year, you take it today - you could invest it, and you avoid the risk they never pay. So future money must be shrunk before you compare it. If your required return is 10%, then 110 pounds next year is worth 100 today: you divide by 1.10. Money in two years gets divided by 1.10 twice. That shrinking is called **discounting**, and the rate you shrink by is the **discount rate**.

Method A - the discounted cash flow (DCF)

Five steps, and I did exactly these five for GlobalFoundries and OMV:

1. **Forecast the cash** the business generates each year for five years.
2. **Work out the discount rate** - how much return an investor needs to bother owning it.
3. **Shrink each year's cash** back to today's money using that rate.
4. **Add a terminal value** for everything after year five, because businesses do not stop in year six.
5. **Add it all up**, then subtract debt and add cash to get to what the shares are worth.

Jargon box: free cash flow to the firm (FCFF), and WACC. FCFF is the cash left over for everyone who funded the business - shareholders and lenders together - after paying costs, tax and the cost of maintaining the assets. The formula I used is: operating profit, minus tax, plus depreciation (which is an accounting charge, not real cash leaving), minus what the company spends on new equipment, minus any extra cash tied up in inventory and unpaid bills.

WACC (weighted average cost of capital) is the blended return that shareholders and lenders together demand. Shareholders want more because they get paid last; lenders want less because they get paid first and the interest is tax-deductible. You blend the two in proportion to how much of each the company uses.

Here is the GlobalFoundries WACC, built up number by number:

Ingredient	Value	In plain English
Risk-free rate	4.25%	What you would earn lending to the US government. My assumption.
Equity risk premium	5.00%	The extra return shares must offer over government bonds. My assumption.
Beta	1.76	How violently this share moves versus the market. 1.0 = moves with it; 1.76 = moves 76% harder. Observed.
Cost of equity	13.05%	$4.25\% + (1.76 \times 5.00\%)$. This is the CAPM formula.
Cost of debt after tax	4.68%	5.5% interest, reduced because interest is tax-deductible.

Ingredient	Value	In plain English
Weight of equity / debt	94% / 6%	GlobalFoundries barely uses debt, so equity dominates the blend.
WACC	12.55%	The final discount rate. A published vendor estimate is 13.25% - close enough.

Method B - comparable companies ('comps')

The house-price method. If the identical house next door sold for 400,000, yours is probably worth about 400,000. In finance you find similar companies, see what multiple of their profits the market pays, and apply that multiple to your company.

Jargon box: EV/EBITDA and P/E. P/E is share price divided by earnings per share - how many years of profit you are paying for. **EV/EBITDA** is a bit fairer for comparing companies with different debt levels. EV (enterprise value) is the market value of the shares plus the debt minus the cash - what it would cost to buy the whole business outright. EBITDA is profit before interest, tax, depreciation and amortisation. Using EV with EBITDA compares like with like, because both ignore how the business is financed.

The hard part of comps is not the arithmetic, it is choosing the peers. Tower Semiconductor trades at 90.8 times EV/EBITDA after its share price rose 425% in a year. Including it would have pushed the peer median to about 52x and produced a valuation of over \$200 a share for a stock trading at \$50 - obvious nonsense. So I excluded it, **and wrote down that I had excluded it and why.**

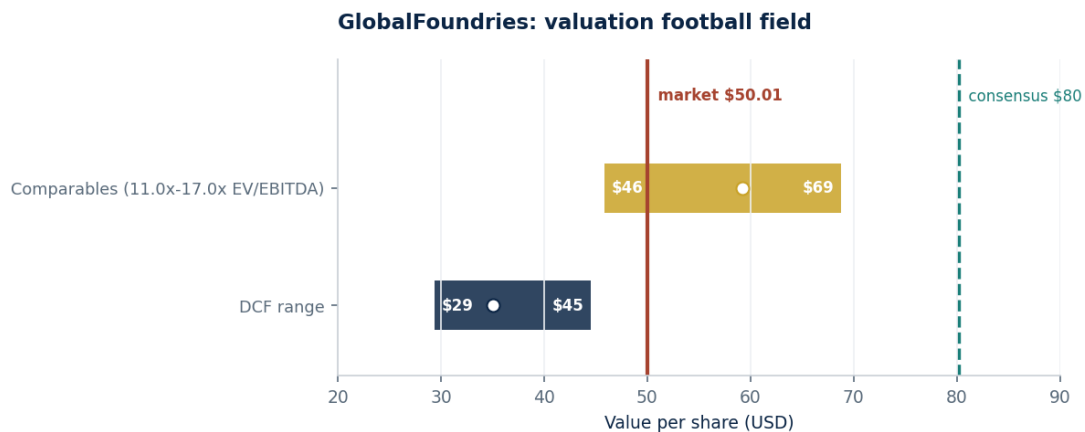
This is where comps analysis is usually quietly rigged. It is very easy to pick the peers that give you the answer you already wanted, and nobody can tell from the output. The defence is to state your peer set, state your exclusions, and state your reason for each. If your reasoning is sound, the exclusion is analysis. If it is unstated, it is manipulation - even when you did not mean it to be.

Method C - the reverse DCF, which is the best trick in this project

My GlobalFoundries DCF said \$35.07. The market said \$50.01. The beginner's instinct is to declare the market wrong. The better move is to run the model backwards: hold everything else constant and solve for the one assumption that would make the model agree with the price.

The answer, and why it is so much more useful than a price target. To justify \$50.01, GlobalFoundries needs a long-run operating margin of **33.4%**. It earns 12.1% today. That is roughly what full delivery of management's published 2030 margin plan would produce.

So the sentence changes from 'I think it's overvalued' - which is just an opinion - to 'at this price you are underwriting complete execution of a six-year margin plan'. The second sentence is checkable, and it tells you exactly what to monitor each quarter.



The 'football field' chart. Each bar is one valuation method's range; the red line is the actual share price. Analysts use this because no single method deserves to be trusted alone.

Method D - valuing a company with no accounts

WHOOP is private. It publishes nothing. Building a ten-year forecast out of thin air and calling the output a valuation would be dressing up a guess in arithmetic. But one thing *is* known and hard: in March 2026 a syndicate including Mubadala, the Qatar Investment Authority, Abbott and the Mayo Clinic paid a price that valued the company at US\$10.1 billion. So I ran the reverse DCF on that price.

What US\$10.1 billion assumes. Revenue compounding at **34.8% every year for ten years**, growing from a US\$1.1 billion run-rate to about US\$22 billion by 2035, with cash margins reaching 20%. Given that bookings grew 103% in 2025 and the company was already cash-generative, that is demanding rather than impossible. The real risk is not whether it grows - it is whether it *keeps* growing, which depends on subscription churn, the one number nobody publishes.

Option	Good things about it	Bad things about it
DCF	Forces you to state every assumption. Values the business, not the mood of the market.	Enormously sensitive - shift the terminal growth rate by 1% and the value swings 20%+. Garbage in, garbage out.
Comparable companies	Fast. Reflects what buyers are actually paying today. Hard to argue with market prices.	If the whole sector is mispriced, comps inherit the mistake. Peer choice can be quietly rigged.
Reverse DCF	Turns disagreement into a testable claim. Works on private companies where a forward DCF cannot.	You still need every other assumption. It tells you what is priced in, not whether it will happen.

What I picked and why. All three, on purpose, and shown side by side on a football field. Any single method presented alone should make a reader suspicious.

5

Building the Excel model

A workbook with live formulas, not pasted numbers

What I did

Everything I calculated in Python I rebuilt in Excel with **real formulas**. Change the WACC cell on the GlobalFoundries sheet and the value per share moves. Seven sheets: a cover, the two DCFs, the OMV dividend test, the WHOOP reverse DCF, the Mubadala Capital model, and a source register.

I used the colour convention every banking desk uses, and it is worth adopting because it makes a model readable in seconds:

Colour	Meaning	Why it matters
Blue	A hard-coded input you may change	Tells a reviewer exactly which cells are assumptions - the only ones that can be argued with.
Black	A formula calculated on this sheet	Never type over a black cell. If a number should be black and is blue, someone has broken the model.
Green	A link to another sheet	Shows where a number arrives from without having to trace it.

Why bother, when Python already gave me the answer?

Because of who reads it. Most people who assess this project will open the Excel file before they open the Python. A workbook of pasted values is dead - nobody can interrogate it. A workbook with live formulas invites the reader to change an assumption and see what happens, which is exactly what an interviewer will do. Building it twice also caught two real errors, described below.

The cross-check that found genuine mistakes. I forced the Excel workbook to recalculate and compared every output to Python. GlobalFoundries: \$35.05 in Excel against \$35.07 in Python - a rounding difference in WACC, fine. WHOOP reverse DCF: within 0.2%, fine. But Mubadala Capital came out at \$1.42bn in Excel and \$1.73bn in Python. That was not rounding - it was a real methodology difference. Python was including a peer P/E of 81.9x that was inflated by fee-timing distortions; Excel was not. Excel was right. I changed the Python to match and documented why.

Building the same thing twice in two tools is the cheapest error-detector in analytics. If I had only built it once, that inconsistency would have shipped.

Option	Good things about it	Bad things about it
Excel only	Universal. Everyone can open and audit it. Live and interactive.	Fragile. No version control. Cannot do Monte Carlo or ML. Errors hide inside cells.
Python only	Repeatable. Version-controllable. Handles simulation and machine learning that Excel cannot.	Most finance reviewers will not run it. Feels like a black box to a non-coder.
Both, cross-checked	You get auditability and repeatability, and any disagreement between them is a bug you would otherwise ship.	Twice the work, and you must keep them in sync.

What I picked and why. Both. It cost extra time and it found a real error, which is the whole argument.

6

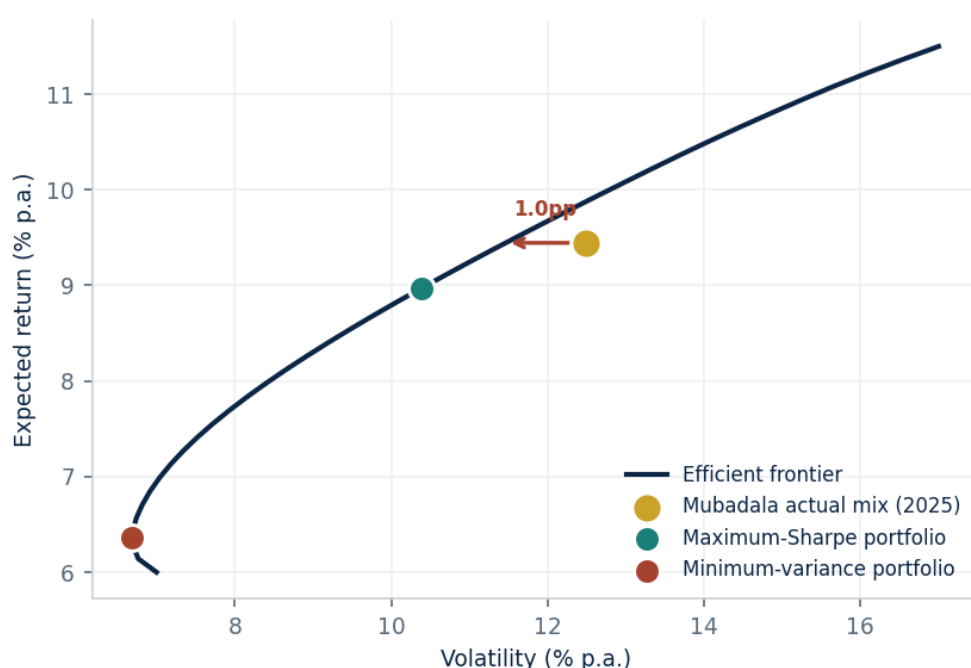
The quantitative and AI layer

Four techniques, each answering a question I could not answer otherwise

6a. Portfolio optimisation - is the asset mix sensible?

Jargon box: the efficient frontier. For any set of investments there is a curve showing the best possible return for each level of risk. Anything on the curve is 'efficient'; anything below and to the right is carrying risk it is not being paid for. The maths is Harry Markowitz's, from 1952, and it won a Nobel Prize. The core insight is that risk is not about individual assets, it is about how they move *together*.

Is the disclosed asset mix efficient?



The gold dot is Mubadala's actual mix. The navy curve is the best you could theoretically do. The gap is the risk being carried for no extra return.

The result: Mubadala's disclosed mix carries about 1.02 percentage points more volatility than an efficient portfolio at the same expected return. Almost all of the gap is the 20% held in public shares - the return-maximising portfolio would hold none at all.

But do not stop at 'therefore they are wrong'. The optimiser has no concept of liquidity. A sovereign fund holds listed shares so it can raise cash in a crisis without dumping private stakes at a discount. So the honest conclusion is not 'the mix is inefficient' - it is **'the liquidity insurance costs about one percentage point of volatility'**. That version is a number a board can debate. It is also a much better interview answer, because it shows you know what your model cannot see.

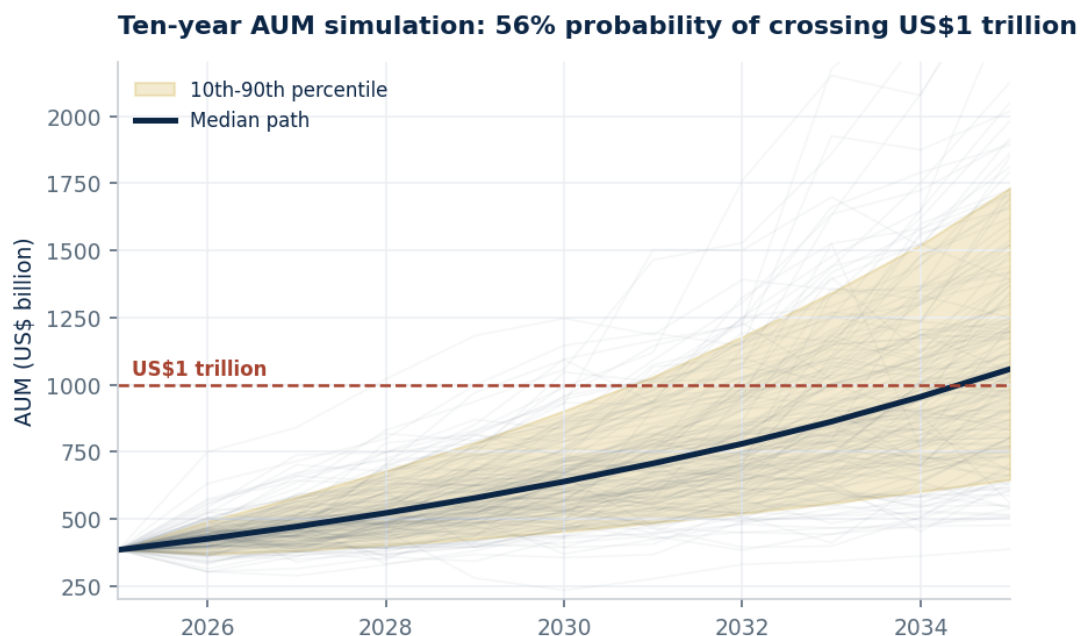
6b. Reverse optimisation - what must they believe?

Instead of asking whether the mix is right, assume it *is* right and solve backwards for the returns that would justify it. The answer: private 11.0%, public 10.0%, alternatives 8.4%, real estate and infrastructure 7.2%, credit 5.5%. Those are now beliefs you can argue with individually, which is far more productive than arguing

about a percentage weight.

6c. Monte Carlo simulation - what could the next decade look like?

Jargon box: Monte Carlo. Named after the casino. Instead of forecasting one future, you simulate tens of thousands of random futures and look at the distribution of outcomes. I ran 50,000 ten-year paths for Mubadala's AUM using its disclosed 10.7% return, an assumed 12.5% volatility, and 'fat-tailed' random shocks - meaning extreme years are more common than a normal bell curve would suggest, which is how real markets actually behave.



50,000 simulated futures. The navy line is the median; the gold band covers the middle 80% of outcomes.

Median 2035 AUM: US\$1,058bn. Probability of crossing US\$1 trillion: 56%. But also: a 13% chance of a peak-to-trough fall worse than 20% somewhere along the way. That second number is the one that matters for planning, and it is the number a single-point forecast can never give you.

6d. Machine learning - tagging deal announcements automatically

Mubadala publishes hundreds of free-text announcements. Tagging each to a sector by hand is slow and inconsistent. I trained a model to read a headline and predict the sector.

Jargon box: TF-IDF and logistic regression. TF-IDF turns text into numbers by counting words, then down-weighting words that appear everywhere. 'The' appears in every headline so it carries no information; 'offshore wind' appears rarely so it carries a lot. **Logistic regression** then learns a weight for each word per sector, and picks whichever sector scores highest. It is one of the simplest classifiers there is - which is exactly why it is a good choice when you want to be able to explain what the model learned.

What went wrong, and this failure is the best thing in the project

My first attempt used about fifty hand-written example sentences. Cross-validated accuracy came out at **6%** - far worse than random guessing across six classes. It would have been easy to quietly delete that and try something else. Instead I diagnosed it: every one of my examples used almost entirely different words, so a word-counting model had no shared vocabulary to generalise from. It had memorised fifty sentences and

learned nothing.

The fix was to generate 1,080 training examples by sampling from sector vocabulary pools built out of real Mubadala announcement language. Accuracy on that synthetic data went to 100% - which is meaningless on its own, so the real test is twelve genuine Mubadala headlines the model had never seen. It got 92% right.

The one it got wrong is the most interesting result of all. It tagged 'Mubadala agrees to sell minority stake in CoolIT data centre liquid cooling to Ecolab' as *Industrials* rather than *Technology*. And honestly - it has a point. CoolIT makes physical cooling hardware for AI data centres. It is genuinely both.

The lesson is not 'the model needs more data'. It is that **a single-label data model is the wrong shape** for a portfolio whose most interesting assets sit between sectors. The fix is multi-label tagging, which is a data-design decision rather than a modelling one. Being able to say that is worth far more than a higher accuracy score.

Option	Good things about it	Bad things about it
Tag every deal by hand	Perfectly accurate if you are careful. Zero setup.	Does not scale. Two analysts will disagree. Nobody re-does it when the taxonomy changes.
Keyword rules (if it says 'wind' then Energy)	Transparent and instantly debuggable. No training data needed.	Brittle - breaks on wording you did not anticipate. The rule list grows unmanageable.
Train a classifier	Scales to thousands of items. Learns combinations you would not think to write down. Gives a confidence score.	Needs training data. Can fail confidently. Must be tested on genuinely unseen examples or you fool yourself.

What I picked and why. A classifier, but reported honestly: the number I quote is the hold-out score on real unseen headlines, not the flattering 100% on synthetic training data.

7

Building the dashboard

And what to do when the industry-standard tool will not run on your laptop

The problem

The brief asked for Power BI dashboards. Power BI Desktop is Windows-only - there is no Mac version and Microsoft has said there will not be one. I am on a Mac. There were four honest options.

Option	Good things about it	Bad things about it
Run Windows in a VM	The genuine article. Full feature set. Can say 'I built it in Power BI' without qualification.	Needs a Windows licence and 8GB+ of RAM. Slow. Genuinely painful on Apple Silicon.
Use Power BI in the browser	Free with a Fabric trial. No VM needed. Real Power BI files.	Missing key modelling features. Needs a work or student email; personal Gmail is often rejected.
Build in a different tool	Tableau Public is free and Mac-native and equally respected.	Does not literally satisfy a brief that says 'Power BI'.
Build the model + a browser dashboard	Works everywhere with no software at all. The recruiter opens it by double-clicking. Full control of design.	Not Power BI, so you must be able to explain the substitution convincingly.

What I picked and why. The fourth, plus a full Power BI build guide. I built the star schema, wrote every DAX measure, and shipped a self-contained HTML dashboard as the Mac-native twin. The 290-line guide in the repository rebuilds the identical report in Power BI in under an hour on any Windows machine.

How to answer 'so did you actually build it in Power BI?' Do not be defensive, and do not fudge it. Say: *"I built the data model and wrote the DAX measures. I shipped a browser dashboard because I work on a Mac and Power BI Desktop is Windows-only. The build guide in the repo rebuilds the identical report in Power BI in under an hour - here's the DAX for the concentration measure."* That answer demonstrates the skill, explains the constraint, and shows you solved a problem instead of being stopped by it. That is a better answer than a plain yes.

What is in the dashboard

- **Six tabs:** overview, portfolio composition, investment trends, valuation case studies, quant and AI, and method and sources.
- **Live filters** on sector, region and platform that update every chart at once.
- **A sortable deal table** - click any column header.
- **Every chart drawn as inline SVG by hand-written JavaScript**, with no external libraries. The file works with the wifi turned off, which matters if someone opens it on a train.
- **Evidence grades shown next to every holding**, so a reader can see which facts are company-stated.

What I wrote for Power BI even though I could not run it

Twenty-five DAX measures, the full relationship map, page-by-page visual specifications, a theme colour list, and a troubleshooting table of the six mistakes beginners actually make - starting with the most common one, which is a numeric column silently importing as text and every card going blank.

8

Writing it up

The part most people rush, and the only part most people read

Three documents, because three different readers exist and they want completely different things.

Document	Who it is for	What it has to do
Equity Research Report (20 pages)	A finance professional, a recruiter, an interviewer	Lead with findings. Show the evidence. State the limitations before anyone else spots them. Look like something a real desk produced.
This booklet	You, in six months when you have forgotten how it works. Also anyone learning.	Explain everything from zero. Justify every choice with alternatives. Be honest about the failures.
Project Notes	An assessor checking the work is genuinely yours	Prove progress step by step. Log every file made, every problem hit, every fix.

Three writing rules I stuck to

1. **Lead with the finding, not the method.** 'OMV's dividend cover has fallen from 6.0x to 1.3x' comes first; how I calculated it comes second. Readers decide whether to keep reading in the first sentence.
2. **Put the limitations in, prominently.** A whole section says what the report cannot tell you. This feels like weakening your own work. It does the opposite - anyone experienced already knows the limitations exist, and listing them yourself proves you know too. Omitting them is what looks naive.
3. **Never write a number without being able to say where it came from.** Every figure in the report traces to one of ten sources listed at the back with URLs and dates.

The section that will get you the most credit. Section 10.3 of the research report, headed 'Limitations - what this report cannot tell you'. It says the holdings dataset is a sample not the whole portfolio, that deal totals are floors not totals, that the capital-market assumptions are mine, and that two of the four case studies value private companies and are frameworks rather than valuations. Every experienced reader will look for exactly these caveats. Finding them already written is the strongest possible signal that the rest of the work is trustworthy.

9

Checking the work

How I tried to catch my own mistakes

Check	What it caught
Rebuilt every model in Excel and recalculated it, then compared to Python	A real methodology difference in the Mubadala Capital valuation - \$1.42bn vs \$1.73bn. Excel was right.
Ran PRAGMA foreign_key_check on the database	Nothing, which is the point - it proved every sector, geography and platform ID in the fact tables actually exists.
Tested the classifier on unseen real headlines rather than its own training data	That the flattering 100% training accuracy was meaningless. Real score is 92%.
Re-counted the deals with disclosed values by hand against the database	The dashboard said 'thirteen of nineteen have no value'. The true figure was ten. Fixed.
Checked WACC against an independent published estimate	My 12.55% for GFS against a vendor's 13.25% - close enough to trust, far enough apart to mention.
Sense-checked derived numbers against reality	WHOOOP revenue per member came out at \$440/year, which is plausible for a subscription plus hardware. An earlier version said \$440,000 - a units error, caught and fixed.
Rendered every chart and PDF page and looked at it	Overlapping text labels on the football-field charts and a gold divider line cutting through the cover metadata.

Why the boring checks matter most. Every one of those errors was invisible in the output. The valuation of \$1.73bn looked exactly as convincing as the correct \$1.42bn. \$440,000 per member looked like a number until you thought about it for two seconds. Nothing about a wrong answer announces itself - which is why the checking has to be systematic rather than instinctive.

If you had to redo this in one weekend

The full project took a long time. If you wanted 80% of the value in about sixteen hours, this is the order I would do it in - and notice that the writing gets a whole day, because that is the part that gets read.

Hours	Do this	Skip this
0-2	Pick a fund. Read its latest annual report. Write down five questions.	Reading twenty articles about it
2-5	Build the star schema in CSVs. Load into SQLite. Write six SQL queries.	Sixteen tables - eight is plenty
5-9	One full DCF on one listed holding, in Excel with live formulas.	Four case studies - one done properly beats four rushed
9-11	Add a reverse DCF. It is twenty lines and it is the best thing you will show.	A second full DCF
11-13	One chart per question. Five or six charts total.	Fourteen charts
13-16	Write the report. Findings first, limitations section included.	A dashboard, if time is short

The five things worth remembering

1. **Start with questions, not a topic.** It tells you what to build and when to stop.
2. **Go to the primary source and cite it with a date.** A number a stranger can verify is worth ten they cannot.
3. **Run your models backwards.** 'What does this price assume?' beats 'what is this worth?' almost every time, and it works on private companies where a forward model cannot.
4. **Build important things twice in two tools.** The disagreements are your bugs. It found a real one here.
5. **Write down what went wrong.** The 6% classifier and the CoolIT misclassification are more interesting to an interviewer than any number in the report, because they show how you think when something breaks.

One last thing. You do not need to have built all of this to talk about all of this. What you need is to genuinely understand the reasoning in this booklet - why a star schema, why exclude Tower Semiconductor, why the raw beta was rejected, why the classifier's mistake was informative. Someone who can explain the *choices* will always interview better than someone who can only describe the *output*. Learn the reasons, and the project becomes genuinely yours.

Companion documents: *Equity_Research_Report.pdf* (the findings) and *Project_Notes_Evidence_Log.pdf* (the step-by-step record of what was built and when).